- *Debugging*: We can set breakpoints, look at variables, memory, registers, stack trace, etc. We can debug only one program at a time, and the debugger stops the program while we do the inspection. *GDB/KDB* is used for such debugging.

So which of these tools would you use? You're probably thinking of using a combination of them; wouldn't it be great to have the capabilities of all these tools combined into one? The response to just such a wish is, SystemTap!

Welcome to SystemTap!

SystemTap can monitor multiple system-wide synchronous and asynchronous events at the same time. It can do scriptable filtering and statistics collection. It's a dynamic method of monitoring and tracing the operations of a running Linux kernel.

To instrument the running kernel, SystemTap uses Kprobes and return probes. With kernel debug information, it gets the addresses for functions and variables referenced in the script. With utrace, SystemTap supports probing user-space executables and shared libraries as well. SystemTap is, therefore, useful to systems administrators, kernel developers, support engineers, researchers and students.

Installation

To install SystemTap on Fedora, run the following code as the root user:

```
yum install systemtap kernel-devel
debuginfo-install kernel
```

To use SystemTap on Ubuntu or any other distro, you need to install the *systemtap* package, and the *debuginfo* packages corresponding to the kernel you're running.

You need to be the root user to run the SystemTap scripts—or you could add a normal user account to either the *stapdev* or *stapusr* groups, to allow that account to run the script.

How does it work?

To understand how SystemTap works, let's run a script in verbose mode (with the -v switch). The *stap* program is the front-end to SystemTap. The *-e* switch instructs it to execute the script in the following argument:

```
$ stap -v -e 'probe syscall.read {printf("syscall %s arguments %s \n",
name, argstr); exit()}'
Pass 1: parsed user script and 65 library script(s) using
83596virt/20428res/2412shr kb, in 150usr/10sys/249real ms.
Pass 2: analyzed script: 1 probe(s), 4 function(s), 0 embed(s), 0 global(s)
using 216260virt/115660res/73964shr kb, in 560usr/20sys/946real ms.
Pass 3: translated to C into "/tmp/stapU0VeZi/stap_b40c8268c87acc68
3f75ded62a52ee66_2113.c" using 216260virt/117180res/75484shr kb, in
320usr/40sys/1014real ms.
Pass 4: compiled C into "stap_b40c8268c87acc683f75ded62a52ee66_2113.ko" in
3010usr/1210sys/12818real ms.
Pass 5: starting run.
syscall read arguments 4, 0x00007fffa773b4c8, 8196
Pass 5: run completed in 20usr/60sys/174real ms.
```

Let's see what happened at each of the passes mentioned:

- Passes 1 and 2: The script we want to run is parsed, and the code is checked for semantic and syntactic errors. Any *tapset* reference is imported. Debug data (provided via *debuginfo* packages) is read to find the addresses for functions and variables referenced in the script.
- Pass 3: The script is translated into C code.
- Pass 4: The translated C code is compiled to create a kernel module.
- Pass 5: The compiled module is inserted into the running kernel.

Once the module is loaded, probes are inserted at proper locations. From now on, whenever a probe is hit, the handler for that probe is called.

The basic syntax we used in our one-line script was to write a *probe* for an *event*, and the *handler* to run when that event occurred:

```
probe <event> { handler }
```

In this syntax:

- *event* is one of the *kernel.function*, *process.statement*, *timer.ms*, *begin*, *end*, *or (tapset)* aliases. For more information, look at the man page for *stapprobes*.
- *handler* can have:
  - filtering/conditionals *(if ... next)*
  - control structures *(foreach, while)*